

Introduction to Algorithms

Email: grad.saeed@gmail.com

Github: github.com/saeedgrad

Analyzing Algorithms

- Analyzing an algorithm has come to mean **predicting the resources that the algorithm requires**. You might consider resources such as **memory, communication bandwidth, or energy consumption**.
- Most often, however, you'll want to measure **computational time**.
- If you analyze several candidate algorithms for a problem, you can identify **the most efficient one**.

Model of the technology

- Before you can analyze an algorithm, you need a **model of the technology** that it runs on, including the resources of that technology and a way to express their costs.
- Most of this book assumes **a generic one-processor, random-access machine (RAM) model** of computation as the implementation technology, with the understanding that algorithms are implemented as computer programs.

Model of the technology

- In the RAM model, **instructions execute one after another**, with no concurrent operations.
- The RAM model assumes that each instruction takes **the same amount of time as any other instruction** and that each data access (using the value of a variable or storing into a variable) takes the same amount of time as any other data access.
- In other words, in the RAM model each **instruction or data access takes a constant amount of time**.

Analysis of insertion sort

- One way to tell would be for you to run it on your computer and time how long it takes to run.
- You would find out **how long insertion sort takes to run on your particular computer, on that particular input, under the particular implementation** that you created, with the particular compiler or interpreter that you ran, with the particular libraries that you linked in, and with the particular background tasks that were running on your computer concurrently with your timing test.

Analysis of insertion sort

- We need a way to **predict**, given a **new input**, how long insertion sort will take.
- Instead of timing a run, or even several runs, of insertion sort, we can determine how long it takes by analyzing the algorithm itself. We'll examine **how many times it executes each line of pseudocode** and **how long each line of pseudocode takes to run**. We'll first come up with a precise but complicated formula for the running time. Then, we'll **distill the important part of the formula** using a convenient notation that can help us compare the running times of different algorithms for the same problem.

The input size

- The running time **depends on the input**.
- Even though the running time can depend on many features of the input, we'll focus on the one that has been shown to have the greatest effect, namely **the size of the input**, and describe the **running time of a program as a function of the size of its input**.
- The best notion for **input size depends on the problem being studied**.

The input size

- For many problems, such as sorting or computing discrete Fourier transforms, the most natural measure is the number of items in the input (for example, **the number n of items being sorted.**)
- For many other problems, such as multiplying two integers, the best measure of input size is **the total number of bits needed to represent the input** in ordinary binary notation.
- For graphs, we usually characterize the input size by **both the number of vertices and the number of edges in the graph.**

The running time

- The running time of an algorithm on a particular input is **the number of instructions and data accesses executed**.
- A constant amount of time is required to execute each line of our pseudocode. One line might take more or less time than another line, but **we'll assume that each execution of the k th line takes c_k time, where c_k is a constant**.
- This viewpoint is in keeping with the RAM model, and it also reflects how the pseudocode would be implemented on most actual computers.

Analysis of insertion sort

github.com/saeedgrad

github.com/saeedgrad

Analysis of insertion sort

INSERTION-SORT(A, n)		<i>cost</i>	<i>times</i>
1	for $i = 2$ to n	c_1	n
2	$key = A[i]$	c_2	$n - 1$
3	<i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4	$j = i - 1$	c_4	$n - 1$
5	while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6	$A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8	$A[j + 1] = key$	c_8	$n - 1$

Analysis of insertion sort

The running time of the algorithm is the sum of running times for each statement executed. A statement that takes c_k steps to execute and executes m times contributes $c_k m$ to the total running time.¹¹ We usually denote the running time of an algorithm on an input of size n by $T(n)$. To compute $T(n)$, the running time of INSERTION-SORT on an input of n values, we sum the products of the *cost* and *times* columns, obtaining

$$\begin{aligned} T(n) = & c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) \\ & + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1) . \end{aligned}$$

Analysis of insertion sort, best-case

Analysis of insertion sort, best-case

Even for inputs of a given size, an algorithm's running time may depend on *which* input of that size is given. For example, in INSERTION-SORT, the best case occurs when the array is already sorted. In this case, each time that line 5 executes, the value of *key*—the value originally in $A[i]$ —is already greater than or equal to all values in $A[1:i-1]$, so that the **while** loop of lines 5–7 always exits upon the first test in line 5. Therefore, we have that $t_i = 1$ for $i = 2, 3, \dots, n$, and the best-case running time is given by

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8). \end{aligned} \tag{2.1}$$

We can express this running time as $an + b$ for *constants* a and b that depend on the statement costs c_k (where $a = c_1 + c_2 + c_4 + c_5 + c_8$ and $b = c_2 + c_4 + c_5 + c_8$). The running time is thus a *linear function* of n .

Analysis of insertion sort, worst-case

Analysis of insertion sort, worst-case

The worst case arises when the array is in reverse sorted order—that is, it starts out in decreasing order. The procedure must compare each element $A[i]$ with each element in the entire sorted subarray $A[1:i-1]$, and so $t_i = i$ for $i = 2, 3, \dots, n$. (The procedure finds that $A[j] > \text{key}$ every time in line 5, and the **while** loop exits only when j reaches 0.) Noting that

$$\begin{aligned}\sum_{i=2}^n i &= \left(\sum_{i=1}^n i \right) - 1 \\ &= \frac{n(n+1)}{2} - 1 \quad (\text{by equation (A.2) on page 1141})\end{aligned}$$

Analysis of insertion sort, worst-case

$$\begin{aligned}\sum_{i=2}^n (i-1) &= \sum_{i=1}^{n-1} i \\ &= \frac{n(n-1)}{2} \quad (\text{again, by equation (A.2)}) ,\end{aligned}$$

we find that in the worst case, the running time of INSERTION-SORT is

$$\begin{aligned}T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ &\quad + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8) .\end{aligned}$$

Analysis of insertion sort, worst-case

We can express this worst-case running time as $an^2 + bn + c$ for constants a , b , and c that again depend on the statement costs c_k (now, $a = c_5/2 + c_6/2 + c_7/2$, $b = c_1 + c_2 + c_4 + c_5/2 - c_6/2 - c_7/2 + c_8$, and $c = -(c_2 + c_4 + c_5 + c_8)$). The running time is thus a *quadratic function* of n .

Worst-case and average-case analysis

github.com/saeedgrad

github.com/saeedgrad

Worst-case and average-case analysis

- The **worst-case running time** of an algorithm gives an **upper bound** on the running time for any input. If you know it, then you have a guarantee that the algorithm never takes any longer.
- The **average-case** is often roughly as bad as the worst case.
- In some particular cases, we'll be interested in the average-case running time of an algorithm. We'll see the technique of **probabilistic analysis** applied to various algorithms throughout this book.

Kinds of analyses

Worst-case: (usually)

- $T(n)$ = maximum time of algorithm on any input of size n .

Average-case: (sometimes)

- $T(n)$ = expected time of algorithm over all inputs of size n .
- Need assumption of statistical distribution of inputs.

Best-case: (bogus)

- Cheat with a slow algorithm that works fast on *some* input.

Order of growth

- Let's now make one more simplifying abstraction: it is the **rate of growth, or order of growth**, of the running time that really interests us.
- We therefore **consider only the leading term of a formula** (e.g., an^2), since the lower-order terms are relatively insignificant for large values of n .
- We also **ignore the leading term's constant coefficient**, since constant factors are less significant than the rate of growth in determining computational efficiency for large inputs.

Order of growth, machine-independent time

What is insertion sort's worst-case time?

- It depends on the speed of our computer:
 - relative speed (on the same machine),
 - absolute speed (on different machines).

BIG IDEA:

- Ignore machine-dependent constants.
- Look at *growth* of $T(n)$ as $n \rightarrow \infty$.

“Asymptotic Analysis”

Θ -notation

Math:

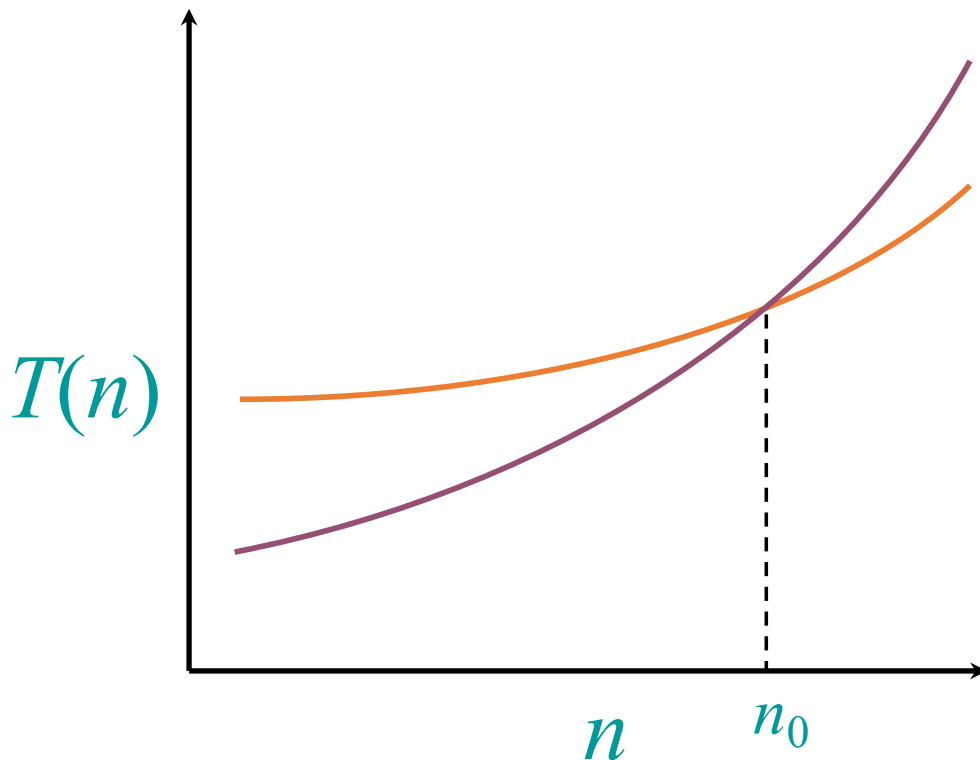
$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \}$

Engineering:

- Drop low-order terms; ignore leading constants.
- Example: $3n^3 + 90n^2 - 5n + 6046 = \Theta(n^3)$

Asymptotic performance

When n gets large enough, a $\Theta(n^2)$ algorithm *always* beats a $\Theta(n^3)$ algorithm.



- We shouldn't ignore asymptotically slower algorithms, however.
- Real-world design situations often call for a careful balancing of engineering objectives.
- Asymptotic analysis is a useful tool to help to structure our thinking.

Insertion sort analysis

Worst case: Input reverse sorted.

$$T(n) = \sum_{j=2}^n \Theta(j) = \Theta(n^2) \quad [\text{arithmetic series}]$$

Average case: All permutations equally likely.

$$T(n) = \sum_{j=2}^n \Theta(j/2) = \Theta(n^2)$$

Is insertion sort a fast sorting algorithm?

- Moderately so, for small n .
- Not at all, for large n .